

# PHP

## Histoire & Evolution

De PHP/FI 1.0 (1994) a PHP 8.3 (2023)

30 ans d'evolution \* Chaque version expliquee \* Nouveautes et exemples

**1994**

Annee de  
creation

**8**

Versions  
majeurs

**30+**

Ans  
d'histoire

**80%**

Sites web  
sous PHP

Un resume complet et comprehensible pour comprendre  
l'evolution du langage PHP a travers ses versions.

**Guide Pedagogique pour Khadija**

2025 - Developpement Web

## Table des Matieres

---

|                     |                                 |      |
|---------------------|---------------------------------|------|
| Introduction        | Qu'est-ce que PHP ?             | p.3  |
| PHP 1.0 / PHP/FI    | 1994–1995 — La naissance        | p.3  |
| PHP 3               | 1997 — La vraie naissance       | p.4  |
| PHP 4               | 2000 — La popularite massive    | p.4  |
| PHP 5               | 2004 — La revolution objet      | p.5  |
| PHP 6               | 2005–2010 — Le projet abandonne | p.6  |
| PHP 7               | 2015 — La renaissance           | p.6  |
| PHP 8.0             | 2020 — La modernite             | p.7  |
| PHP 8.1             | 2021 — Enumerations & Fibres    | p.8  |
| PHP 8.2             | 2022 — Classes en lecture seule | p.9  |
| PHP 8.3             | 2023 — Constantes typees        | p.9  |
| Comparaison globale | Tableau recapitulatif           | p.10 |

## Qu'est-ce que PHP ?

PHP (Hypertext Preprocessor) est un **langage de programmation cote serveur** conçu spécialement pour le développement web. Contrairement au JavaScript qui s'exécute dans le navigateur, PHP s'exécute sur le **serveur** et génère du HTML qui est ensuite envoyé au navigateur.

| Caractéristique | Valeur                                                     | Date |
|-----------------|------------------------------------------------------------|------|
| Cree par        | Rasmus Lerdorf                                             | 1994 |
| Signifie        | Personal Home Page Tools puis Hypertext Preprocessor       |      |
| Type            | Langage serveur, interprete, dynamique                     |      |
| Utilise par     | WordPress, Facebook (debut), Wikipedia, Drupal, Laravel... |      |
| Part de marche  | ~77% des sites web avec un langage serveur connu (2024)    |      |

### PHP/FI 1.0 & 2.0

1994 — 1997 | *La Naissance*

Rasmus Lerdorf crée des scripts Perl pour surveiller les visites de son CV en ligne. Ces scripts deviennent 'Personal Home Page Tools' (PHP Tools), puis PHP/FI (Forms Interpreter). C'est le point de départ d'un des langages les plus utilisés au monde.

- **Scripts Perl convertis en C**  
Rasmus recrit ses scripts en C pour plus de performance. Ce n'était pas prévu pour devenir un vrai langage.
- **Traitement de formulaires HTML**  
PHP/FI permet de traiter les données envoyées via des formulaires web — une révolution pour l'époque.
- **Connexion aux bases de données**  
■ Support basique de mSQL (mini SQL), permettant de lire/écrire des données en base.
- **Variables et boucles simples**  
Syntaxe très basique : variables (\$var), conditions IF, boucles WHILE. Pas de fonctions complexes.

■ **Anecdote** : PHP/FI n'était pas prévu pour devenir un langage. C'était juste des outils personnels de Rasmus. Il les a partagés publiquement en 1994 et la communauté a commencé à les utiliser et améliorer.

## PHP 3

1997 | *La Vraie Naissance de PHP*

Zeev Suraski et Andi Gutmans reecrivent complètement PHP/FI pour créer PHP 3. C'est la version qui donne son nom définitif : PHP devient officiellement 'PHP: Hypertext Preprocessor'. Le langage prend forme.

- **Reecriture complete du moteur**  
Zeev et Andi reecrivent le parseur de zéro. Le code est plus propre, plus extensible, et beaucoup plus stable.
- **Système d'extensions**  
Introduction d'une architecture de modules : on peut ajouter des extensions (MySQL, PostgreSQL, GD...) sans toucher au cœur.
- **Meilleur support objet**  
Première forme de programmation orientée objet (classes basiques). Très limitée mais c'est un début.
- **Adoption massive**  
PHP 3 est adopté par des milliers de sites. C'est ici que PHP devient populaire.
- **Support multi-bases de données**  
MySQL, PostgreSQL, Oracle... PHP 3 supporte plusieurs bases de données simultanément.

```
<?php
// PHP 3 - Exemple basique
class Animal {
    var $nom;

    function Animal($n) { $this->nom = $n; }

    function parler() { echo $this->nom . ' fait un bruit'; }
}

$chat = new Animal('Felix');
$chat->parler(); // Felix fait un bruit
?>
```

## PHP 4

2000 — 2008 | *La Popularité Massive*

PHP 4 introduit le moteur Zend Engine 1.0, cree par Zeev Suraski et Andi Gutmans. C'est la version qui a fait exploser PHP dans le monde entier. WordPress, Joomla, et des milliers de CMS naissent avec PHP 4.

#### ■ **Zend Engine 1.0**

Nouveau moteur d'execution beaucoup plus rapide. C'est le coeur de PHP qui sera ameliore dans toutes les versions suivantes.

#### ■ **Sessions HTTP**

Gestion native des sessions : on peut maintenant garder l'utilisateur connecte entre les pages (session\_start(), \$\_SESSION).

#### ■ **Bufferisation de sortie**

ob\_start() permet de capturer la sortie HTML avant de l'envoyer. Essentiel pour les redirections et la mise en cache.

#### ■ **Fonctions de securite**

Ajout de md5(), sha1(), crypt() pour le hashage de mots de passe (bien que pas suffisant par la suite).

#### ■ **Support HTTP ameliore**

Meilleur acces aux variables superglobales : \$\_GET, \$\_POST, \$\_COOKIE, \$\_SERVER, \$\_FILES.

#### ■ **Bibliotheque de fonctions**

Des centaines de fonctions integrees : manipulation de chaines, tableaux, fichiers, images (GD), PDF...

```
<?php
// PHP 4 - Sessions et superglobales
session_start();
$_SESSION['utilisateur'] = 'Khadija';

// Recuperer donnees formulaire
$nom = $_POST['nom'];
$email = $_GET['email'];

// Hashage (basique - ne pas utiliser en prod aujourd'hui)
$mdp_hash = md5('mon_mot_de_passe');
?>
```

■ PHP 4 est sorti en 2000 et a ete officiellement abandonne en 2008. Mais beaucoup de sites anciens tournaient encore sous PHP 4 en 2015 ! C'est pourquoi la securite etait un gros probleme.

## PHP 5

2004 — 2018 | *La Revolution Orientee Objet*

PHP 5 avec le Zend Engine 2.0 est une revolution majeure. La POO (Programmation Orientee Objet) devient vraiment utilisable. C'est la version qui a defini PHP moderne. La plupart des grands frameworks (Laravel, Symfony) sont nes avec PHP 5.

### ■ POO Complete (OOP)

■ Vraie programmation orientee objet : interfaces, classes abstraites, heritage, visibilite (public/private/protected), exceptions.

### ■ PHP Data Objects (PDO)

Interface unifiee pour acceder a n'importe quelle base de donnees. Plus besoin de fonctions differentes pour MySQL/PostgreSQL.

### ■ Gestion d'exceptions (try/catch)

Systeme de gestion d'erreurs moderne avec try, catch, finally. Fini les erreurs fatales non gerees.

### ■ SPL (Standard PHP Library)

Ensemble de structures de donnees et d'iterateurs : ArrayObject, SplStack, SplQueue, SplHeap...

### ■ Methodes magiques

\_\_construct(), \_\_destruct(), \_\_get(), \_\_set(), \_\_toString(), \_\_call()... automatisent des comportements.

### ■ SimpleXML & DOM

Manipulation facile du XML et HTML. Lecture, ecriture et navigation dans des documents XML.

### ■ Filtrage des donnees

filter\_var() pour valider et nettoyer les entrees utilisateurs (emails, URLs, entiers...).

### ■ PHP 5.3 : Namespaces & Closures

PHP 5.3 (2009) ajoute les namespaces (eviter les conflits de noms) et les fonctions anonymes (closures).

### ■ PHP 5.4 : Traits

Les traits permettent de reutiliser du code dans plusieurs classes sans heritage.

### ■ PHP 5.5 : Generators & OPcache

Les generateurs (yield) permettent d'iterer sur de grands ensembles sans tout charger en memoire.

<?php

```
// PHP 5 - POO Complete

interface Forme {
    public function aire(): float;
}

abstract class FigureGeometrique implements Forme {
    protected string $couleur;
    public function __construct(string $couleur) {
        $this->couleur = $couleur;
    }
}

class Cercle extends FigureGeometrique {
    public function __construct(private float $rayon, string $couleur) {
        parent::__construct($couleur);
    }
    public function aire(): float {
        return M_PI * $this->rayon ** 2;
    }
}

try {
    $c = new Cercle(5.0, 'rouge');
    echo 'Aire : ' . $c->aire(); // Aire : 78.53...
} catch (Exception $e) {
    echo 'Erreur : ' . $e->getMessage();
}

// Closures (PHP 5.3+)
$multiplier = fn($x) => $x * 2;
echo $multiplier(5); // 10

// PDO - Acces base de donnees unifie
$pdo = new PDO('mysql:host=localhost;dbname=test', 'user', 'pass');
$stmt = $pdo->prepare('SELECT * FROM users WHERE id = ?');
$stmt->execute([1]);

?>
```

## PHP 6

2005 — 2010 | *Le Projet Abandonne*

PHP 6 est un cas unique dans l'histoire de PHP : c'est la seule version majeure qui n'a JAMAIS été publiée officiellement. Le projet a été abandonné après 5 ans de développement. Une leçon importante sur la gestion de projets complexes.

- **Objectif : Support Unicode natif**  
L'idée principale était d'intégrer Unicode (UTF-16) dans le cœur de PHP pour mieux gérer les langues du monde entier.
- **Problèmes de performance catastrophiques**  
Convertir toutes les chaînes en UTF-16 interne ralentissait énormément le moteur. Les benchmarks étaient terribles.
- **Difficulté d'implémentation**  
Des centaines d'extensions devaient être mises à jour pour supporter Unicode. La tâche était gigantesque.
- **Abandon officiel en 2010**  
L'équipe de développement annonce officiellement l'abandon de PHP 6. Les fonctionnalités utiles seront intégrées dans PHP 5.3 et 5.4.

✗ PHP 6 n'existe pas dans les releases officielles. Si tu vois un livre ou un cours 'PHP 6', sache qu'il est basé sur un projet abandonné et n'est donc pas fiable. Le numéro de version 6 a été sauté et on est passé directement à PHP 7.

## PHP 7

2015 — 2022 | *La Renaissance*

PHP 7 est une révolution ! Avec le nouveau moteur PHPNG (PHP Next Generation) / Zend Engine 3.0, PHP 7 est jusqu'à 2x plus rapide que PHP 5.6. C'est la version qui a sauvé la réputation de PHP face à Node.js et Python.

- **Performance x2 (Zend Engine 3.0)**  
Le moteur complètement réécrit exécute le code jusqu'à 2 fois plus vite que PHP 5.6, avec une consommation mémoire réduite de moitié.
- **Déclaration de types scalaires**  
On peut maintenant indiquer le type attendu des paramètres et valeurs de retour : int, float, string, bool.
- **Opérateur Null Coalescing (??)**  
`$val = $_GET['name'] ?? 'default';` // Plus besoin de isset() partout !

### ■ Operateur Spaceship (<=>)

Compare deux valeurs : retourne -1, 0 ou 1. Parfait pour les tris personnalisés.

### ■ Classes anonymes

Créer une classe sans nom à la volée : `new class { ... }`. Utile pour les tests et les callbacks.

### ■ Typage strict (declare strict\_types=1)

Mode strict : PHP vérifie que les types passés sont exactement corrects, sans conversion automatique.

### ■ PHP 7.1 : Types nullable & void

?string = peut être une string OU null. void = la fonction ne retourne rien.

### ■ PHP 7.4 : Propriétés typées & Arrow Functions

`class User { public int $age; }` et `fn($x) => $x * 2` (syntaxe courte pour closures).

```
<?php
declare(strict_types=1);

// PHP 7 - Typage et nouveautés
function additionner(int $a, int $b): int {
    return $a + $b;
}

// Null coalescing
$nom = $_GET['nom'] ?? 'Anonyme';

// Spaceship operator
usort($tableau, fn($a, $b) => $a['age'] <=> $b['age']);

// Classes anonymes
$objet = new class('Khadija') {
    public function __construct(public string $nom) {}
    public function saluer(): string { return 'Bonjour ' . $this->nom; }
};
echo $objet->saluer(); // Bonjour Khadija

// PHP 7.4 - Propriétés typées
class Utilisateur {
    public int $id;
    public string $nom;
    public ?string $email = null; // nullable
```

```
}  
?>
```

**PHP 8.0***Novembre 2020 | L'Ere Moderne*

PHP 8.0 est une version majeure avec de nombreuses fonctionnalités très attendues. Le JIT (Just-In-Time compilation) est la star de cette version, mais les Named Arguments, Attributes et Match expressions changent la façon d'écrire PHP.

### ■ **JIT - Just In Time Compilation**

PHP peut maintenant compiler le code en bytecode machine à la volée. Enorme gain pour les calculs mathématiques et algorithmes complexes.

### ■ **Named Arguments (Arguments nommés)**

■ Passer les arguments par nom : `array_slice(array: $arr, offset: 0, length: 3)`. L'ordre n'a plus d'importance.

### # **Attributes (Annotations natives)**

■ `#[Route('/home')]` au lieu de `/** @Route('/home') */`. Les métadonnées sont désormais natives au langage.

### ■ **Match Expression**

Comme switch mais plus puissant : compare en strict (`===`), retourne une valeur, pas de break nécessaire.

### ■ **Nullsafe Operator (?.)**

`$pays = $user?->getAdresse()?->getPays();` Si `$user` est null, retourne null sans erreur.

### ■ **Constructor Property Promotion**

■ Plus besoin de déclarer les propriétés puis les assigner dans le constructeur. Tout en une ligne.

### ■ **Union Types**

`function test(int|string $val)` : Un paramètre peut accepter plusieurs types.

### ■ **str\_contains(), str\_starts\_with(), str\_ends\_with()**

Fonctions très demandées pendant des années ! Vérifier si une chaîne contient/commence/fini par une autre.

```
<?php
// PHP 8.0 - Les grandes nouveautés

// Constructor Property Promotion
class Produit {
    public function __construct(
        public readonly string $nom,
        public float $prix,
```

```
public int $stock = 0
) {}
}

// Match expression (replace switch)
$statut = 'actif';
$message = match($statut) {
    'actif' => 'Compte actif',
    'inactif' => 'Compte inactif',
    'banni' => 'Compte banni',
    default => 'Statut inconnu',
};

// Nullsafe operator
$ville = $user?->getAdresse()?->getVille() ?? 'Non renseignee';

// Named arguments
echo str_pad(string: 'PHP', length: 10, pad_string: '*', pad_type:
STR_PAD_BOTH);

// Nouvelles fonctions string
echo str_contains('Bonjour Khadija', 'Khadija'); // true
echo str_starts_with('PHP 8.0', 'PHP'); // true
?>
```

## PHP 8.1

Novembre 2021 | Enumerations & Fibres

PHP 8.1 apporte les Enumerations (Enums) tant attendues, les Fibres pour la programmation asynchrone, et les propriétés readonly. Une version riche qui rapproche PHP des langages modernes comme TypeScript.

### ■ Enumerations (Enums)

Définir un ensemble fini de valeurs possibles. Remplace les constantes dispersées. Peuvent être backed (string/int) ou pures.

### ■ Fibres (Fibers)

Mécanisme de concurrence coopérative : suspendre et reprendre l'exécution d'une fonction. Base pour l'asynchrone en PHP.

### ■ Propriétés Readonly

Une propriété readonly ne peut être assignée qu'une seule fois (dans le constructeur). Parfait pour les objets immuables.

### ■ Intersection Types

fonction test(Stringable&Countable; \$val) : le paramètre doit implémenter les DEUX interfaces.

### ■ never Return Type

Indique qu'une fonction ne retourne jamais (throw ou boucle infinie). Aide les outils d'analyse statique.

### ■ Array Unpacking with String Keys

\$merged = [...\$array1, ...\$array2]; fonctionne maintenant avec des clés de type string.

### ■ new in Initializers

```
class Logger { public function __construct(private Handler $h = new NullHandler()) {} }
```

```
<?php
// PHP 8.1 - Enums

enum Statut: string {
    case Actif = 'actif';
    case Inactif = 'inactif';
    case Banni = 'banni';

    public function label(): string {
        return match($this) {
            Statut::Actif => 'Compte actif',
            Statut::Inactif => 'Compte suspendu',
```

```
Statut::Banni => 'Compte banni',  
};  
}  
}  
  
$s = Statut::Actif;  
echo $s->value; // actif  
echo $s->label(); // Compte actif  
  
// Readonly property  
class Commande {  
    public function __construct(  
        public readonly int $id,  
        public readonly string $reference,  
    ) {}  
}  
  
$cmd = new Commande(1, 'CMD-001');  
// $cmd->id = 2; // ERREUR : propriete readonly !  
?>
```

## PHP 8.2

Novembre 2022 | *Classes Readonly & DNF Types*

PHP 8.2 continue la modernisation avec les classes readonly completes, les types DNF (Disjunctive Normal Form), et la deprecation de certaines fonctionnalites dangereuses. Une version de consolidation.

### ■ **Classes Readonly completes**

Tout une classe peut etre declaree readonly : toutes ses proprietes sont automatiquement readonly.

### ■ **DNF Types (Disjunctive Normal Form)**

Combiner Union et Intersection : (Stringable&Countable;)|null. Tres flexible pour le typage avance.

### ■ **Constantes dans les Traits**

Les traits peuvent maintenant definir des constantes, pas seulement des methodes et proprietes.

### ■ **Deprecation de \${} dans les strings**

echo "Bonjour \${nom}"; est deprecie. Utiliser {\$nom} ou la concatenation.

### ■ **Sensible Parameter (masquage)**

#[\SensitiveParameter] : masque la valeur d'un parametre dans les stack traces (ex: mot de passe).

### ■ **Ameliorations de performance**

Optimisations internes du moteur Zend pour reduire la consommation memoire.

```
<?php
// PHP 8.2 - Readonly class
readonly class Point {
    public function __construct(
        public float $x,
        public float $y,
        public float $z = 0.0
    ) {}
}

$p = new Point(1.0, 2.5, 3.0);
echo $p->x; // 1.0
// $p->x = 5.0; // ERREUR - toute la classe est readonly

// DNF Types
```

```
function traiter((Stringable&Countable)|null $val): void {
    if ($val === null) return;
    echo count($val) . ' elements : ' . (string)$val;
}

// SensitiveParameter
function connexion(
    string $hote,
    #[\SensitiveParameter] string $motDePasse
): void {
    // Si erreur, motDePasse sera masque dans le log
}

?>
```

## PHP 8.3

Novembre 2023 | Constantes Typees & JSON Validate

PHP 8.3 apporte des ameliorations ciblées : constantes de classe typees, json\_validate() tres attendu, Override attribute, et des ameliorations de la classe Randomizer pour la generation aleatoire.

### ■ Constantes de classe typees

■ interface Config { const string VERSION = '1.0'; } Les constantes peuvent maintenant avoir un type explicite.

### ■ json\_validate()

Valider du JSON sans le parser entierement. Plus rapide que json\_decode() + verification d'erreur.

### # Attribut #[Override]

■ Indique explicitement qu'une methode remplace une methode parent. Erreur si la methode parente n'existe pas.

### ■ Randomizer ameliorie

Nouvelles methodes : getBytesFromString(), getFloat(), nextFloat(). Meilleure generation aleatoire cryptographique.

### ■ Acces dynamique aux constantes de classe

\$classe::{'CONSTANTE\_' . \$nom} : acceder a une constante dont le nom est dynamique.

### ■ Améliorations stack trace

Meilleure lisibilité des erreurs et exceptions. Les messages sont plus clairs et plus utiles.

```
<?php
// PHP 8.3 - Constantes typees
interface Configurable {
    const string VERSION = '1.0';
    const int MAX_RETRIES = 3;
    const float TIMEOUT = 30.0;
}

// json_validate() - Tres utile !
$json_valide = json_validate('{ "nom": "Khadija" }'); // true
$json_invalide = json_validate('{ nom: Khadija }'); // false

if (!json_validate($donnees_recues)) {
    throw new InvalidArgumentException('JSON invalide');
}

$data = json_decode($donnees_recues, true);

// Attribut Override
class Base {
    public function calculer(): int { return 42; }
}

class Enfant extends Base {
    #[Override]
    public function calculer(): int { return 100; }
    // #[Override] sur une methode inexistante dans Base = ERREUR !
}

?>
```

## Tableau Comparatif de toutes les versions

| Version    | Annee | Moteur           | Innovation majeure                   | Statut        |
|------------|-------|------------------|--------------------------------------|---------------|
| PHP/FI 1.0 | 1994  | Perl/C           | Traitement formulaires web           | Fin de vie    |
| PHP/FI 2.0 | 1997  | C                | Support MySQL, variables             | Fin de vie    |
| PHP 3      | 1997  | Zend 1.0 (beta)  | Architecture extensible, OOP basique | Fin de vie    |
| PHP 4      | 2000  | Zend 1.0         | Sessions, superglobales, popularite  | Fin de vie    |
| PHP 5      | 2004  | Zend 2.0         | OOP complete, PDO, exceptions        | Fin de vie    |
| PHP 6      | 2010  | —                | Projet abandonne (Unicode UTF-16)    | ANNULE        |
| PHP 7.0    | 2015  | Zend 3.0 (PHPNG) | x2 performances, types scalaires     | Fin de vie    |
| PHP 7.4    | 2019  | Zend 3.0         | Proprietes typees, arrow functions   | Fin de vie    |
| PHP 8.0    | 2020  | Zend 4.0 + JIT   | JIT, Match, Named args, Attributes   | Fin de vie    |
| PHP 8.1    | 2021  | Zend 4.0         | Enums, Fibres, Readonly              | Fin de vie    |
| PHP 8.2    | 2022  | Zend 4.0         | Readonly classes, DNF Types          | Securite only |
| PHP 8.3    | 2023  | Zend 4.0         | Constantes typees, json_validate     | Actif         |

## Ce qu'il faut retenir

### 1994-1999 : Les fondations



PHP naît par accident, comme outil personnel. La communaute le transforme en langage.

### 2000-2014 : La domination



PHP 4 et 5 conquèrent le web. WordPress, Joomla, Drupal font de PHP LE langage du web.

### 2015-2019 : La renaissance



PHP 7 répond à Node.js et Python avec des performances doublées. PHP survit et prospère.

### 2020-aujourd'hui : La modernité

PHP 8.x est un langage moderne, type, rapide et expressif. Comparable aux meilleurs langages.

*PHP 8.3 est la version à utiliser aujourd'hui. Si tu commences un projet PHP, utilise PHP 8.1+ minimum. Evite PHP 5 et 7 qui ne recevront plus de correctifs de sécurité. Bonne chance dans tes projets PHP, Khadija*

!